

Voyage Analytics – Integrating MLOps in Travel

Submitted By – Akshat Mishra

GitHub Link - [Voyage-Analytics-Integrating-MLOps-in-Travel : GitHub](#)

1. Project Title

Voyage Analytics – Integrating MLOps in Travel

2. Project Overview

Voyage Analytics is an end-to-end MLOps-based travel analytics system designed to automate machine learning workflows and provide intelligent travel-related predictions and recommendations.

The project integrates:

- Machine Learning Models
- Flask REST APIs
- Streamlit Dashboard
- Docker Containerization
- Kubernetes Deployment
- Apache Airflow Automation
- Jenkins CI/CD Pipeline
- MLflow Experiment Tracking

The system utilizes travel datasets containing user, flight, and hotel information to:

- Predict flight prices
 - Predict user gender
 - Recommend hotels
 - Automate ML workflows
 - Deploy scalable ML services
-

3. Business Problem

In the travel and tourism industry, large volumes of user, hotel, and flight data are generated daily. Businesses need intelligent systems that can:

- Predict flight prices accurately
- Understand user demographics
- Recommend hotels based on preferences
- Automate retraining and deployment workflows
- Deploy scalable machine learning applications

Traditional systems often lack:

- automation
- scalability
- deployment consistency
- workflow orchestration

This project solves these challenges using MLOps principles.

4. Project Objectives

The main objectives of the project are:

1. Build a regression model for flight price prediction.
 2. Build a classification model for gender prediction.
 3. Build a recommendation system for hotel suggestions.
 4. Develop REST APIs using Flask.
 5. Containerize applications using Docker.
 6. Deploy applications using Kubernetes.
 7. Automate workflows using Apache Airflow.
 8. Track experiments using MLflow.
 9. Implement CI/CD using Jenkins.
 10. Build an interactive Streamlit dashboard.
-

5. Technologies Used

Technology	Purpose
Python	Programming Language

Technology	Purpose
Pandas	Data Processing
NumPy	Numerical Computation
Scikit-learn	Machine Learning
Flask	REST API Development
Streamlit	Dashboard Development
Docker	Containerization
Kubernetes	Container Orchestration
Apache Airflow	Workflow Automation
Jenkins	CI/CD Automation
MLflow	Experiment Tracking
Git & GitHub	Version Control
Postman	API Testing

6. Dataset Description

6.1 Users Dataset

Contains user demographic details.

Column	Description
code	User identifier
company	Associated company
name	User name
gender	Gender
age	Age

6.2 Flights Dataset

Contains flight booking details.

Column	Description
travelCode	Travel identifier
userCode	User identifier
from	Source location
to	Destination location

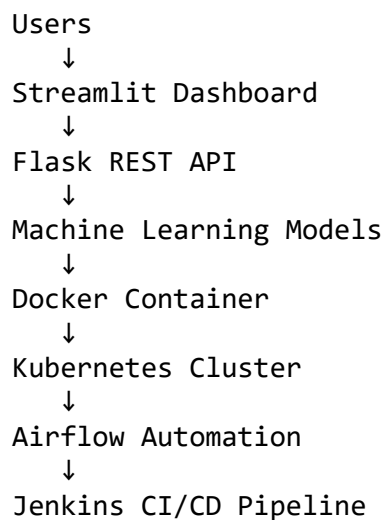
Column	Description
flightType	Flight class
price	Flight price
time	Travel duration
distance	Flight distance
agency	Airline agency
date	Booking date

6.3 Hotels Dataset

Contains hotel booking information.

Column	Description
travelCode	Travel identifier
userCode	User identifier
name	Hotel name
place	Hotel location
days	Stay duration
price	Price per day
total	Total booking cost
date	Booking date

7. Project Architecture



8. Project Folder Structure

VoyageAnalytics/

```
├── api/
├── airflow/
├── classification_model/
├── data/
├── jenkins/
├── kubernetes/
├── models/
├── notebooks/
├── recommendation_model/
├── regression_model/
├── streamlit_app/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

9. Data Preprocessing

The following preprocessing steps were performed:

- Removed duplicate records
 - Handled missing values
 - Converted date columns
 - Extracted year, month, and day features
 - Encoded categorical variables using LabelEncoder
 - Split datasets into training and testing sets
-

10. Exploratory Data Analysis (EDA)

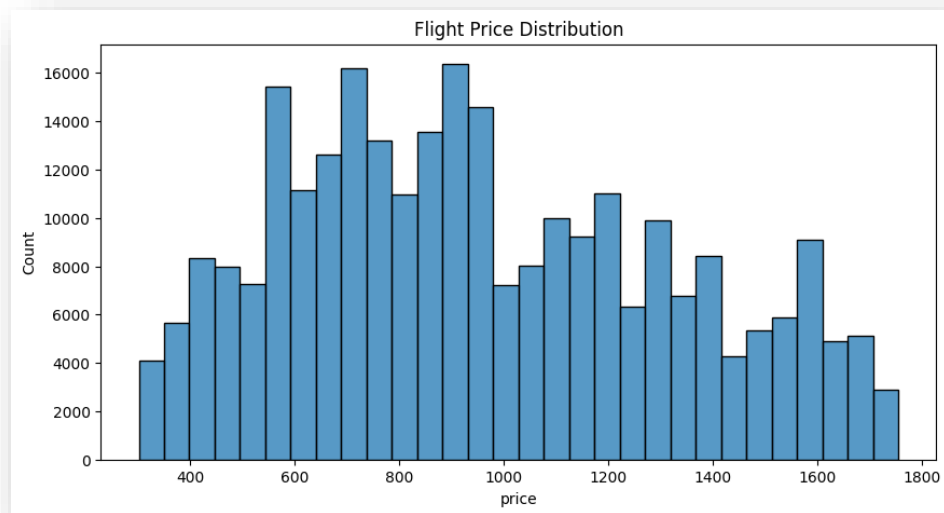
The following visualizations were created:

- Flight price distribution
- Flight type analysis
- Top airline agencies
- Most common destinations
- Gender distribution
- Age distribution

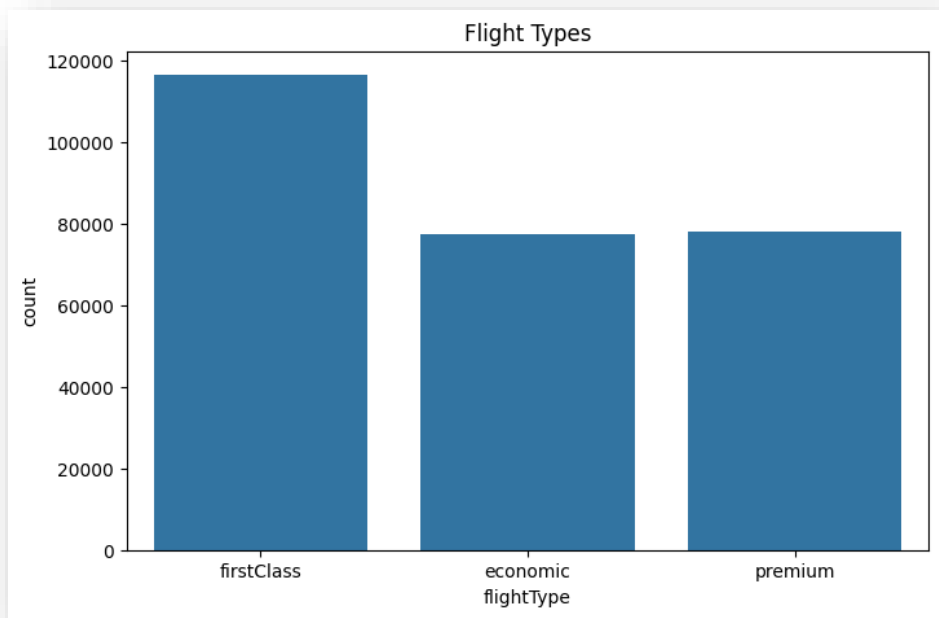
- Hotel price distribution
 - Popular hotel locations
 - Correlation heatmaps
-

SCREENSHOT SECTION

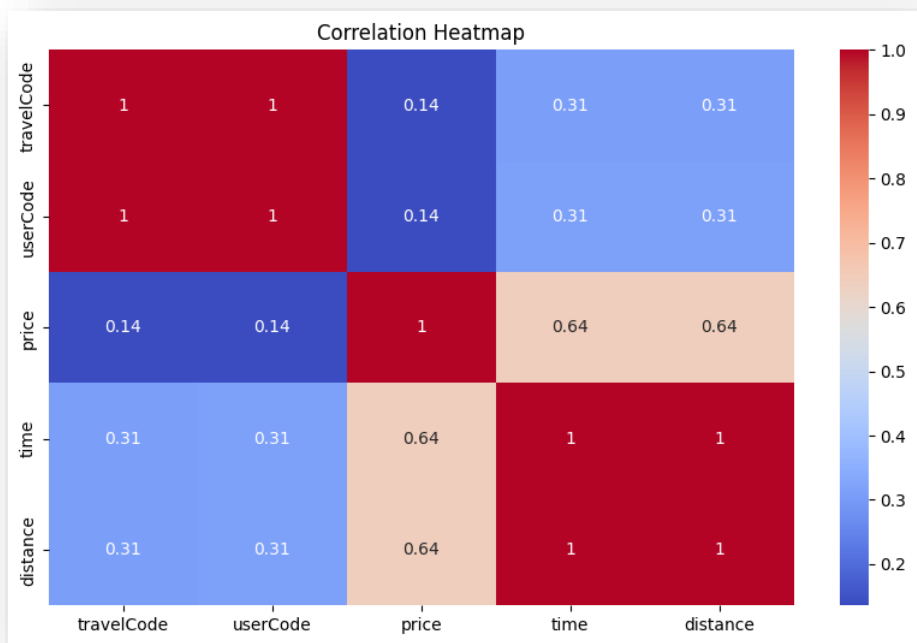
Screenshot 1 – Flight Price Distribution



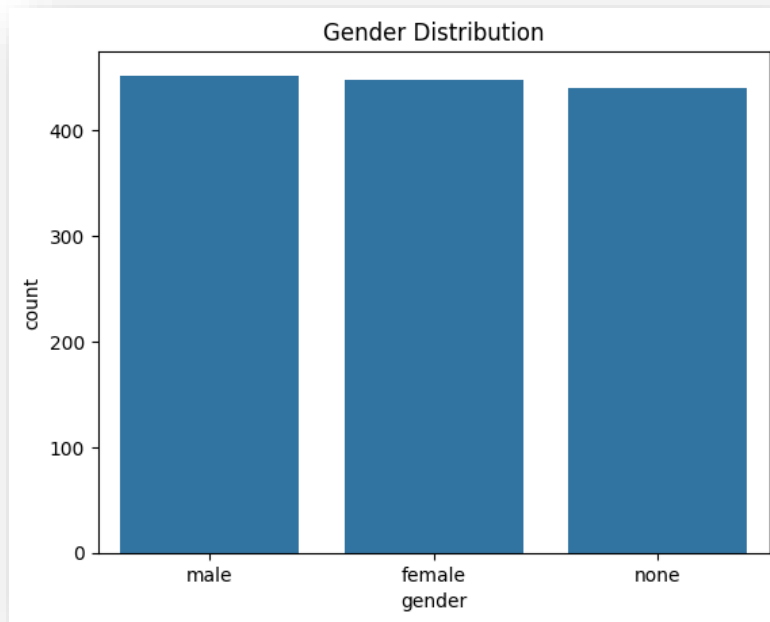
Screenshot 2 – Flight Type Count



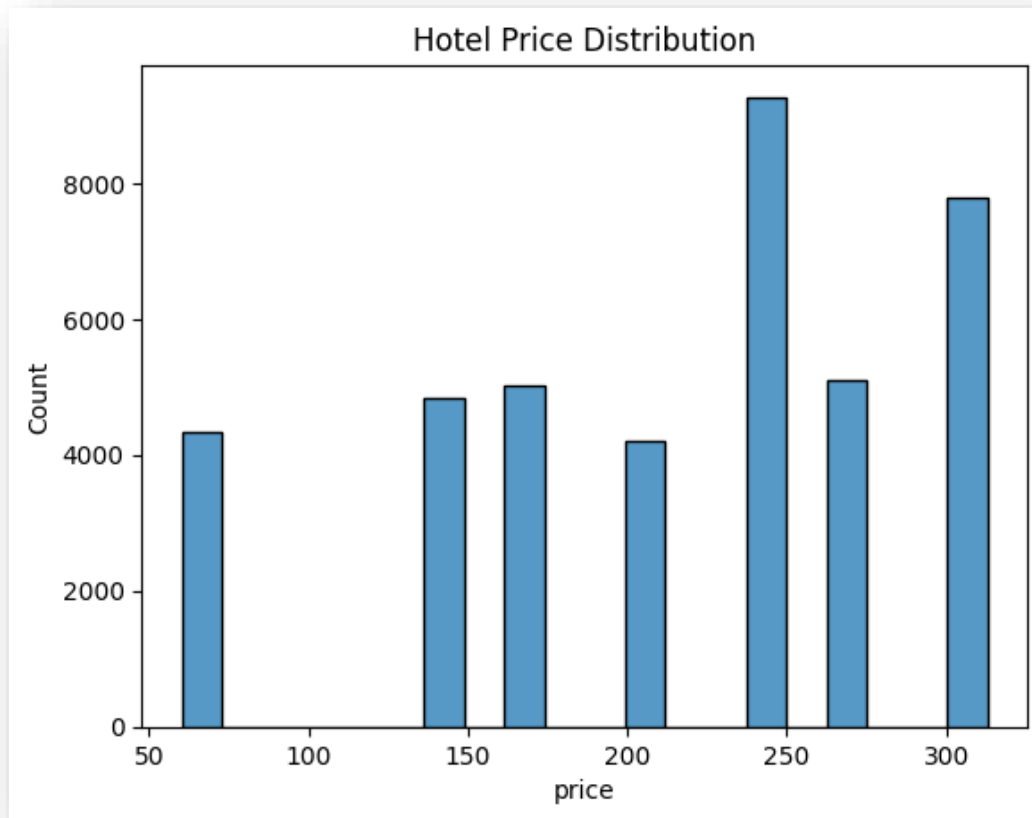
Screenshot 3 – Correlation Heatmap



Screenshot 4 – Gender Distribution



Screenshot 5 – Hotel Price Distribution



11. Regression Model – Flight Price Prediction

Objective

Predict flight prices based on:

- distance
- time
- flight type
- agency
- source and destination

Model Used

RandomForestRegressor

Features Used

- from
- to
- distance
- time
- agency
- flightType
- year
- month
- day

Evaluation Metrics

- MAE
- MSE
- R2 Score

SCREENSHOT SECTION

Screenshot 6 – Regression Metrics Output

The screenshot displays the mlflow web interface for a model named 'industrious-bat-280'. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for 'GenAI', 'Model training', 'Runs', 'Models', 'Traces', 'Model registry', 'Assistant', 'Docs', and 'Settings'.
- Header:** Shows the mlflow logo and version '3.12.0'.
- Breadcrumb:** 'Flight_Price_Regression > Runs > industrious-bat-280'.
- Overview Tab:** The active tab, showing a description (No description), metrics, and parameters.
- Metrics (3):** A table with 3 columns: Metric, Value, and Models.
- Parameters (2):** A section for parameters, currently empty.
- About this run:** A section on the right providing details about the run, including creation time, creator, experiment ID, status, run ID, duration, source, and registered prompts.
- Datasets:** A section showing no datasets are associated with this run.
- Tags:** A section with a link to 'Add tags'.

Metric	Value	Models
MAE	0.07145953326995301	flight_price_model
MSE	0.5684989771108762	flight_price_model
R2_Score	0.9999956856461076	flight_price_model

About this run

Created at	05/10/2026, 10:18:55 PM
Created by	dell
Experiment ID	420911347125532591
Status	Finished
Run ID	d4bcd26026094f45ad259e06a3f9225a
Duration	1.0min
Source	train_regression.py
Registered prompts	—

Datasets

None

Tags

[Add tags](#)

12. Classification Model – Gender Prediction

Objective

Predict user gender using:

- company
- age
- name

Model Used

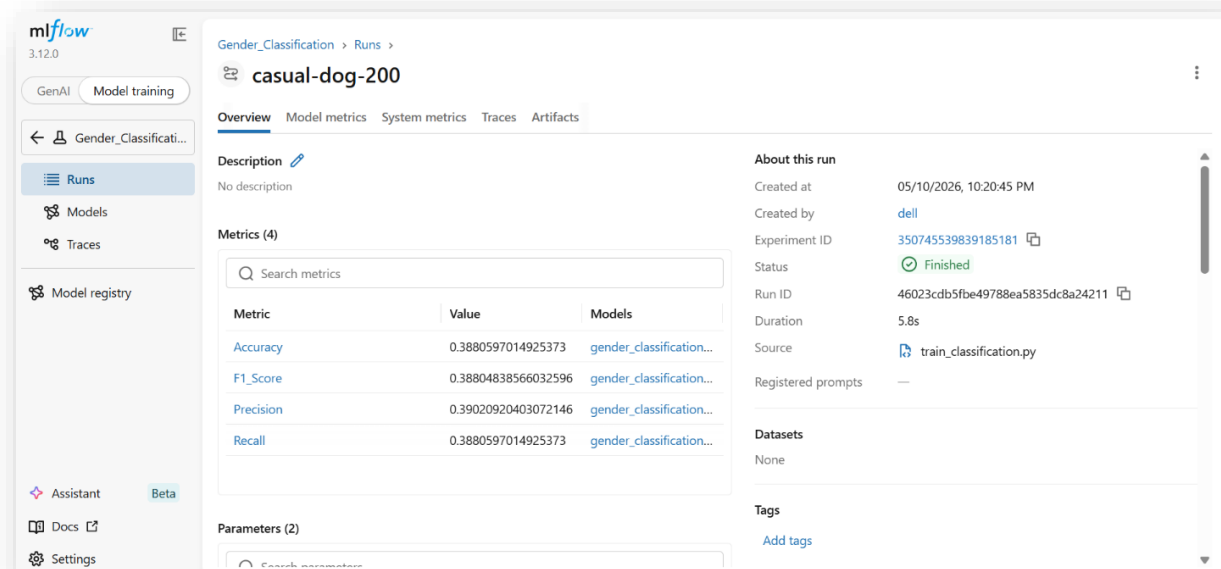
RandomForestClassifier

Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

SCREENSHOT SECTION

Screenshot 9 – Classification Metrics Output



13. Recommendation System

Objective

Recommend hotels based on user preferences and hotel similarity.

Technique Used

Content-Based Recommendation System

Algorithm Used

- CountVectorizer
- Cosine Similarity

Features Used

- hotel name
 - place
 - stay duration
 - price
-

14. MLflow Integration

MLflow was used for:

- experiment tracking
- parameter logging
- metric logging
- model versioning

Logged Parameters

- number of estimators
- random state

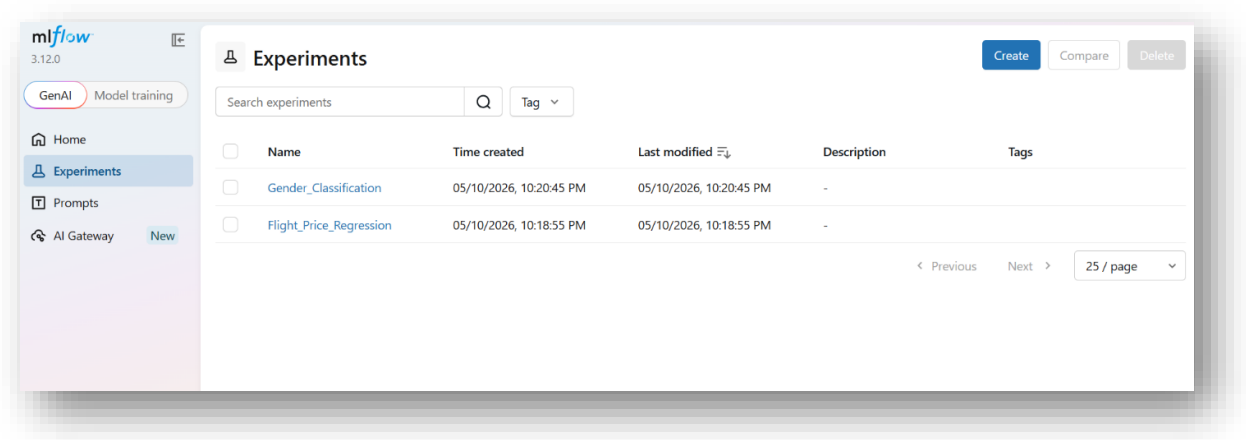
Logged Metrics

- MAE
- MSE
- R2 Score
- Accuracy

- Precision
- Recall
- F1 Score

SCREENSHOT SECTION

Screenshot 13 – MLflow Dashboard



15. Flask REST API

APIs Developed

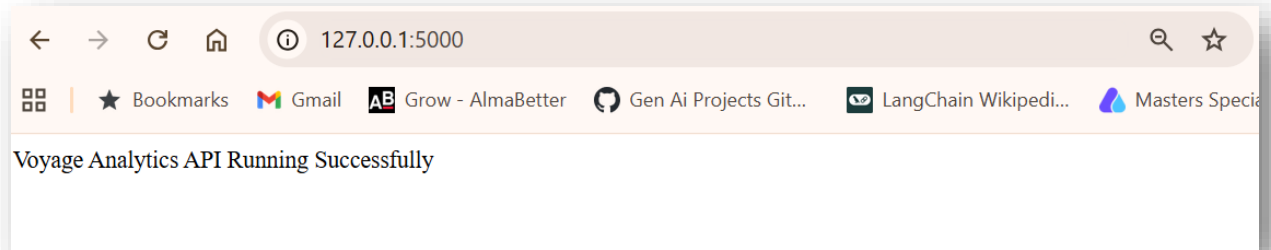
Endpoint	Purpose
/	Home route
/predict-flight-price	Flight prediction
/predict-gender	Gender prediction
/recommend-hotels	Hotel recommendation

Features

- JSON-based API responses
- Real-time predictions
- Recommendation support
- API testing through Postman

SCREENSHOT SECTION

Screenshot 15 – Flask API Running



16. Streamlit Dashboard

Features Included

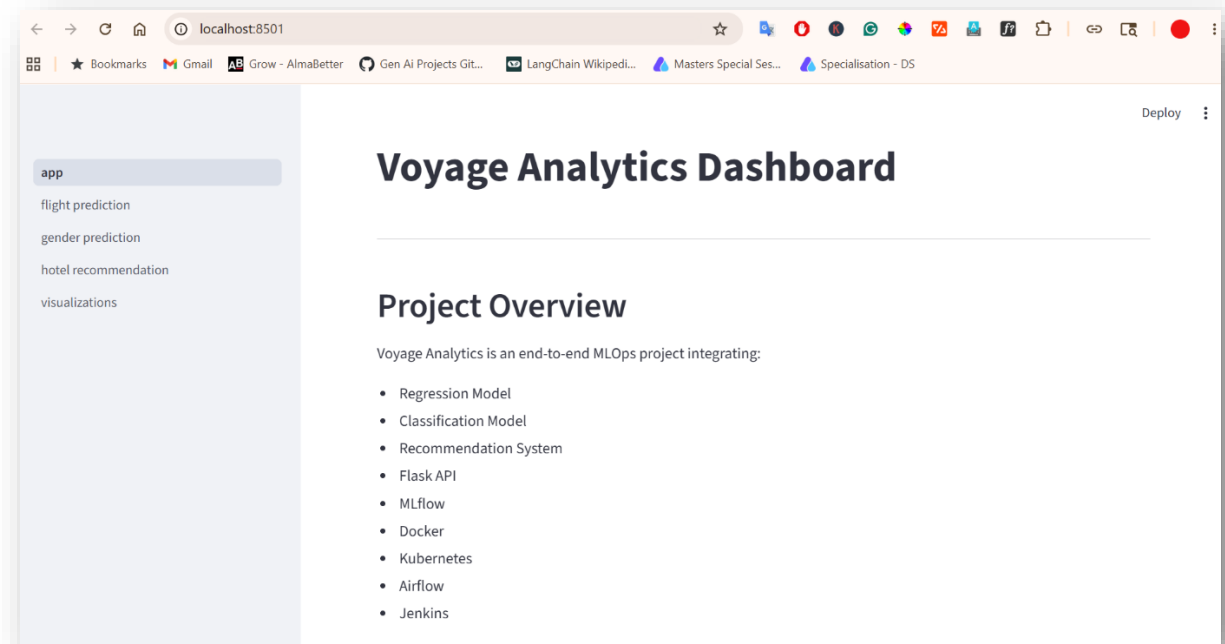
- Project Overview
- Flight Price Prediction
- Gender Prediction
- Hotel Recommendation
- Visualizations

Benefits

- Interactive UI
 - User-friendly interface
 - Real-time API integration
-

SCREENSHOT SECTION

Screenshot 19 – Streamlit Home Page



Screenshot 20 – Flight Prediction Page

← → ↻ 🏠 ⓘ localhost:8501/flight_prediction

🗄️ | ★ Bookmarks 📧 Gmail 📄 AB Grow - AlmaBetter 🔄 Gen Ai Projects Git...

app

flight prediction

gender prediction

hotel recommendation

visualizations

Flight Price Prediction

From Location

0

To Location

0

Distance

0.00

Flight Time

0.00

Agency

0.00

Flight Type

0.00

Year

2025

Month

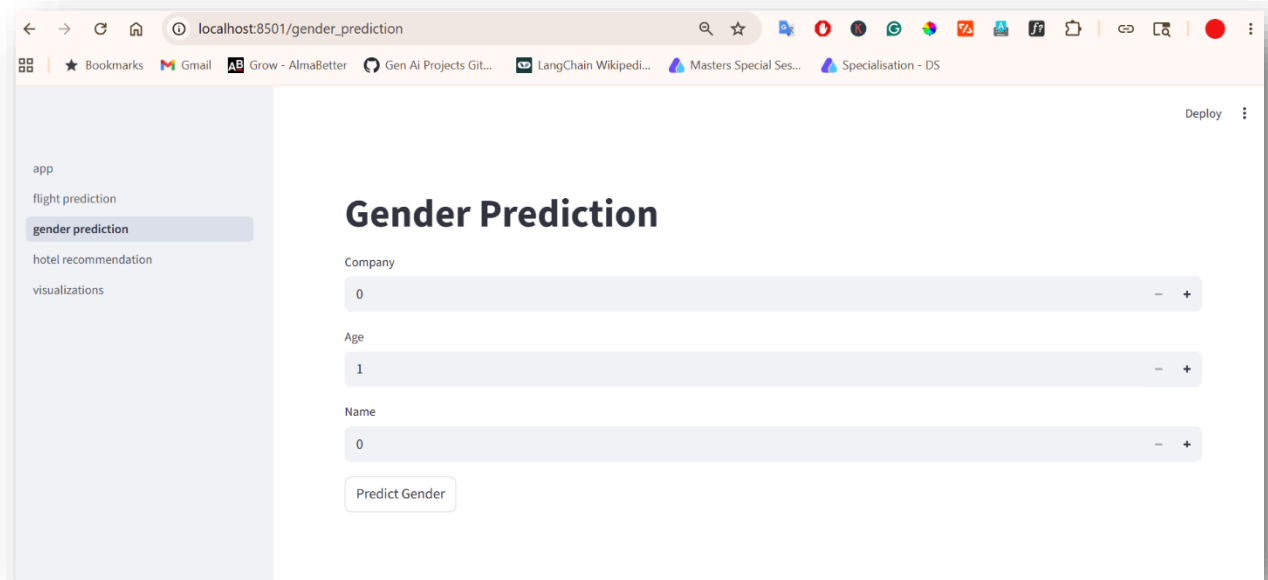
5

Day

10

Predict Flight Price

Screenshot 21 – Gender Prediction Page



17. Docker Containerization

Docker was used to package the Flask API and dependencies into a portable container.

Docker Components

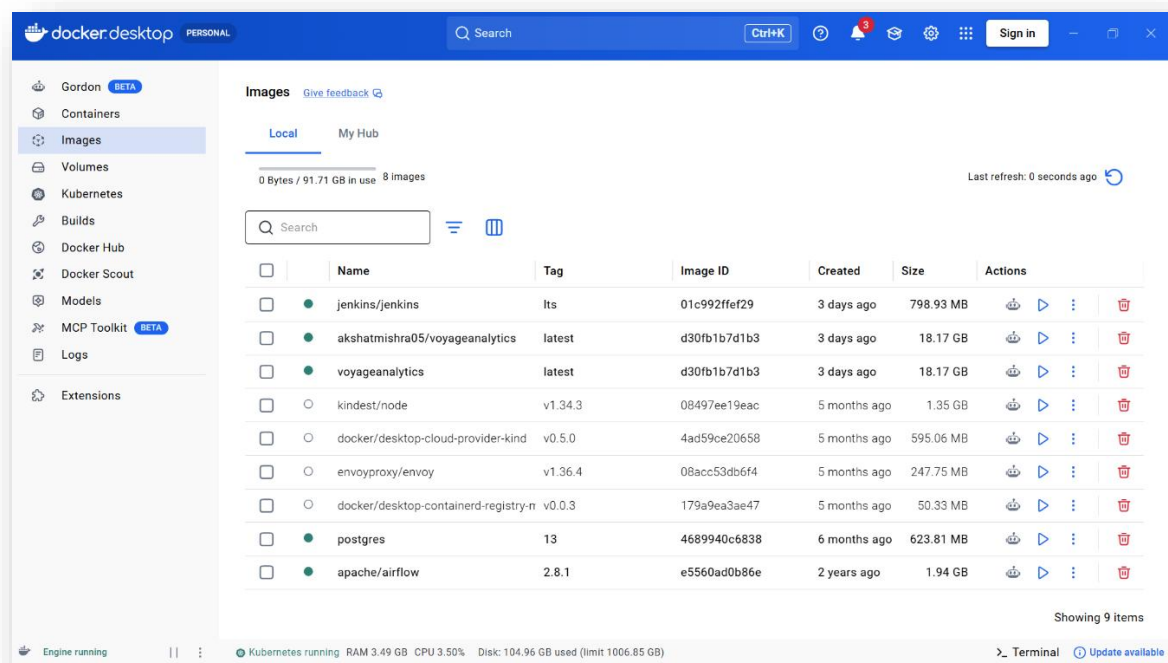
- Dockerfile
- docker-compose.yml
- DockerHub image

Benefits

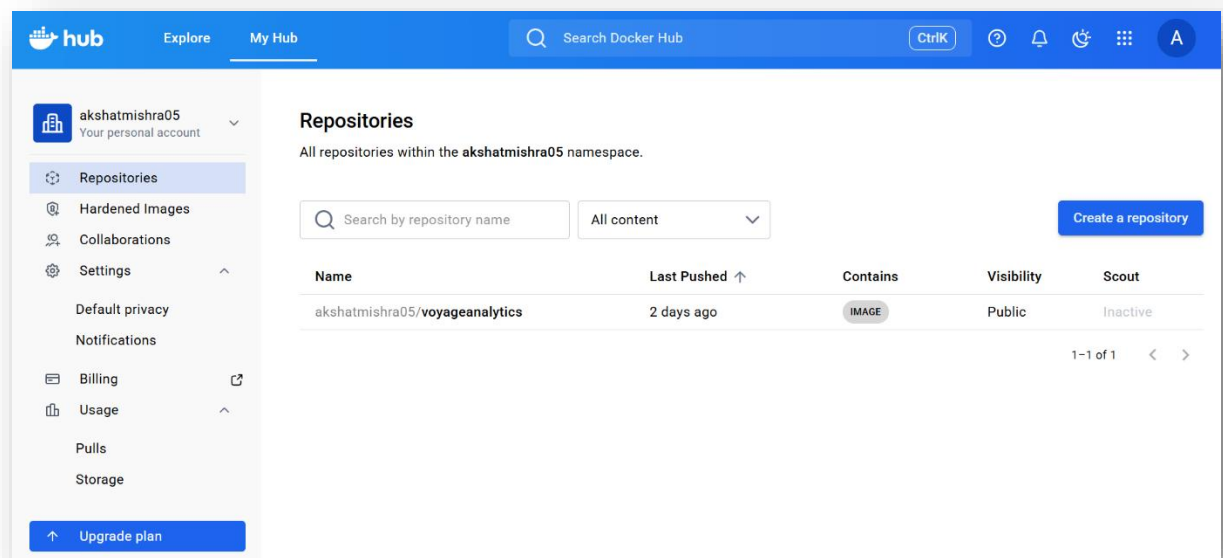
- portability
 - consistency
 - deployment simplicity
-

SCREENSHOT SECTION

Screenshot 23 – Docker Images



Screenshot 25 – DockerHub Repository



18. Kubernetes Deployment

Kubernetes was used for:

- deployment management
- scaling
- load balancing
- orchestration

Kubernetes Files

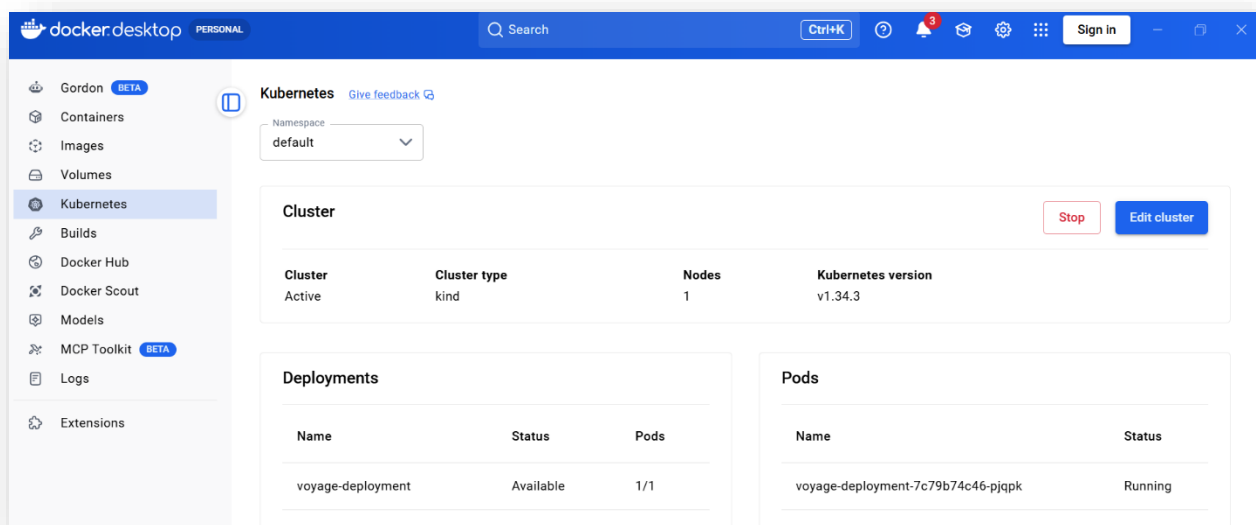
- deployment.yaml
- service.yaml

Features

- multiple replicas
 - service exposure
 - scalable deployment
-

SCREENSHOT SECTION

Screenshot 26 – Kubernetes Pods, Service & Deployment



19. Apache Airflow Integration

Apache Airflow was used for workflow orchestration.

Workflow Tasks

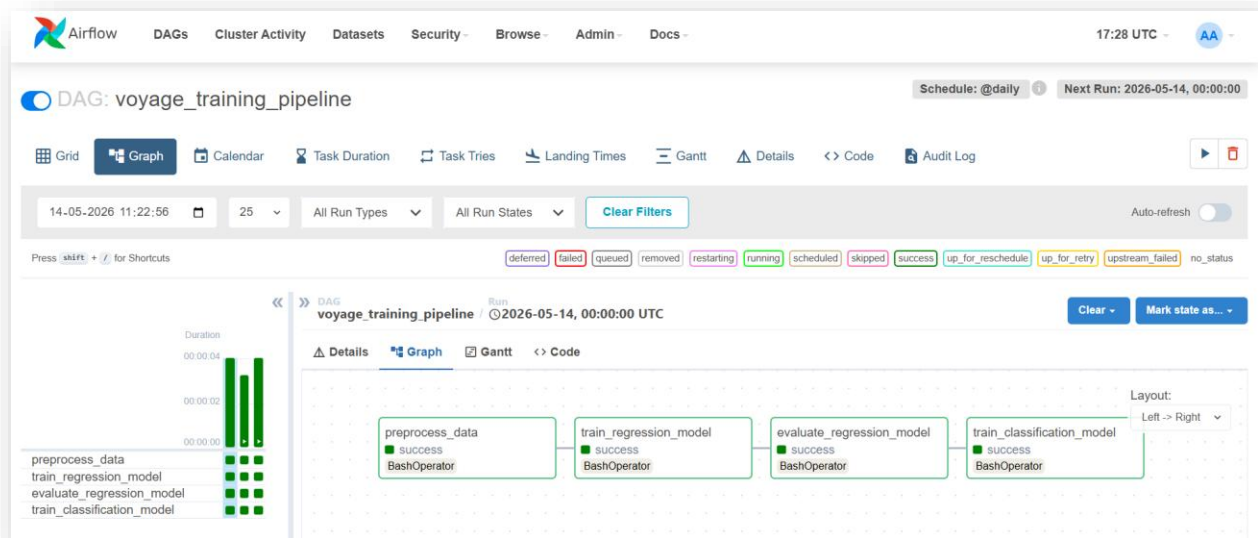
- preprocess data
- train regression model
- evaluate regression model
- train classification model

DAG Features

- automated execution
 - workflow dependency management
 - scheduling support
-

SCREENSHOT SECTION

Screenshot 29 – Airflow Dashboard, DAG Graph



20. Jenkins CI/CD Pipeline

Jenkins was used for:

- continuous integration
- automated builds
- Docker image creation
- deployment automation

Pipeline Stages

1. Clone Repository
2. Install Dependencies
3. Run Training Scripts
4. Build Docker Image

SCREENSHOT SECTION

Screenshot 32 – Jenkins Dashboard, Successful Pipeline

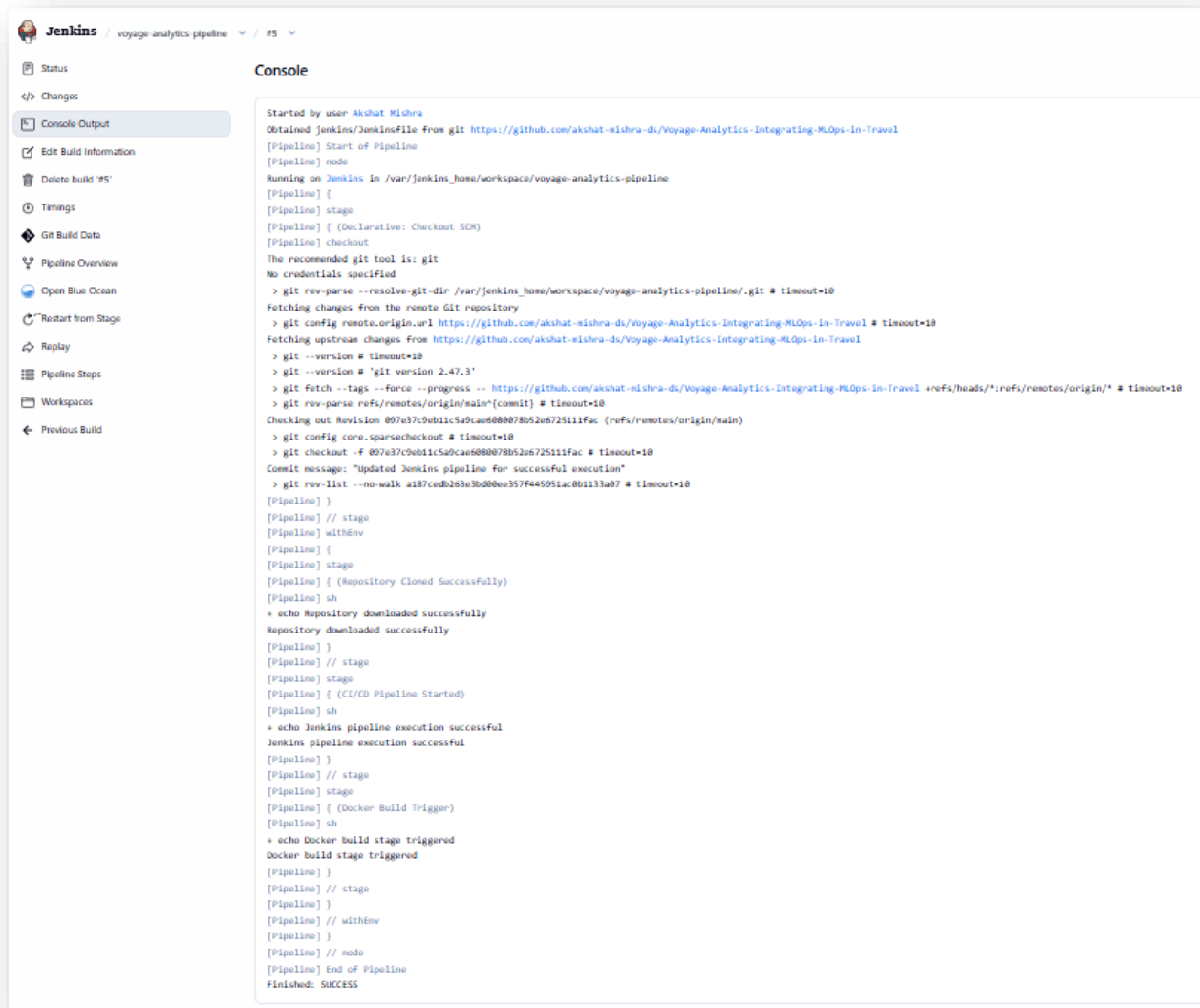
The screenshot shows the Jenkins dashboard for a pipeline named 'voyage-analytics-pipeline'. The pipeline is in a successful state, indicated by a green checkmark. The 'Stage View' section displays a table of stage times for the current build (#5).

Stage	Declarative: Checkout SCM	Repository Cloned Successfully	CI/CD Pipeline Started	Docker Build Trigger
Average stage times: (full run time: ~5s)	1s	461ms	441ms	409ms
#5 May 14 22:25 1 commit	1s	461ms	441ms	409ms

The 'Permalinks' section lists several links for the current build and its history.

- Last build (#5), 1 min 52 sec ago
- Last stable build (#5), 1 min 52 sec ago
- Last successful build (#5), 1 min 52 sec ago
- Last failed build (#4), 5 min 12 sec ago
- Last unsuccessful build (#4), 5 min 12 sec ago
- Last completed build (#5), 1 min 52 sec ago

Screenshot 34 – Jenkins Console Output



The screenshot shows the Jenkins web interface for a pipeline named 'voyage-analytics-pipeline'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Edit Build Information, Delete build #5, Timings, Git Build Data, Pipeline Overview, Open Blue Ocean, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main area displays the console output for build #5, which shows the pipeline's execution steps and their results.

```
Started by user Akshat Mishra
Obtained Jenkins/Jenkinsfile from git https://github.com/akshat-mishra-ds/Voyage-Analytics-Integrating-MLOps-In-Travel
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/voyage-analytics-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git
No credentials specified
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/voyage-analytics-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/akshat-mishra-ds/Voyage-Analytics-Integrating-MLOps-In-Travel # timeout=10
Fetching upstream changes from https://github.com/akshat-mishra-ds/Voyage-Analytics-Integrating-MLOps-In-Travel
> git --version # timeout=10
> git fetch --tags --force --progress -- https://github.com/akshat-mishra-ds/Voyage-Analytics-Integrating-MLOps-In-Travel +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 897a37c9eb11c5a9cae888078b52a672511fac (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 897a37c9eb11c5a9cae888078b52a672511fac # timeout=10
Commit message: "Updated Jenkins pipeline for successful execution"
> git rev-list --no-walk a187cedb283e3bd8be357f445951ac8b1133a87 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Repository Cloned Successfully)
[Pipeline] sh
+ echo Repository downloaded successfully
Repository downloaded successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (CI/CD Pipeline Started)
[Pipeline] sh
+ echo Jenkins pipeline execution successful
Jenkins pipeline execution successful
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Docker Build Trigger)
[Pipeline] sh
+ echo Docker build stage triggered
Docker build stage triggered
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

21. Challenges Faced

Challenge	Solution
Import path issues	Executed project from root folder
Kubernetes image pull issues	Used DockerHub image
Flask import errors	Fixed module paths
Airflow setup complexity	Used Docker-based Airflow setup

22. Key Learnings

During this project the following skills were learned:

- end-to-end MLOps workflow
 - model deployment
 - Docker containerization
 - Kubernetes orchestration
 - workflow automation
 - CI/CD pipelines
 - API development
 - recommendation systems
 - MLflow tracking
-

23. Future Improvements

Future enhancements can include:

- cloud deployment on AWS/GCP/Azure
 - database integration
 - authentication system
 - advanced recommendation algorithms
 - model monitoring dashboards
 - automated retraining
 - FastAPI implementation
 - deep learning models
-

24. Conclusion

Voyage Analytics successfully demonstrates the integration of machine learning and MLOps technologies into a unified travel analytics platform.

The project achieved:

- predictive analytics
- recommendation capabilities

- workflow automation
- scalable deployment
- CI/CD automation
- container orchestration

This project provides a strong foundation for real-world production-grade machine learning systems.

25. References

- [Python Documentation](#)
 - [Scikit-learn Documentation](#)
 - [Flask Documentation](#)
 - [Streamlit Documentation](#)
 - [Docker Documentation](#)
 - [Kubernetes Documentation](#)
 - [Apache Airflow Documentation](#)
 - [Jenkins Documentation](#)
 - [MLflow Documentation](#)
-